DSA

Lab Exercise — Binary Search Trees

Q. A BST satisfies one rule at every node: all keys in the left subtree are smaller than the node's key, and all keys in the right subtree are larger. Duplicates are not allowed in this lab; ignore a duplicate insert.

Define the Node structure yourself: an integer key, a pointer to the left child, and a pointer to the right child.

- **bst_lab.c**

  Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>

// Define the BST Node structure
typedef struct Node {
    int key;
    struct Node* left;
    struct Node* right;
} Node;

// PART A: Build the Tree

// A1. Allocate a new node with malloc
Node* createNode(int key) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->key = key;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// A2. Recursive insertion (ignores duplicates)
Node* insert(Node* root, int key) {
    // Base case: found an empty spot, create the node here
    if (root == NULL) {
        return createNode(key);
```

```
    }

    // Otherwise, recur down the tree
    if (key < root->key) {
        root->left = insert(root->left, key);
    } else if (key > root->key) {
        root->right = insert(root->right, key);
    }
    // Note: If key == root->key, we do nothing (duplicates are ignored)

    return root;
}

// A3. Iterative search (returns 1 if found, 0 if not)
int search(Node* root, int key) {
    Node* current = root;
    while (current != NULL) {
        if (key == current->key) {
            return 1; // Found the key
        } else if (key < current->key) {
            current = current->left; // Go to left subtree
        } else {
            current = current->right; // Go to right subtree
        }
    }
    return 0; // Reached NULL, key is not in the tree
}

// PART B: Traversals and Measurements

// B1. Depth-First Traversals
void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    printf("%d ", root->key);
    inorder(root->right);
}

void preorder(Node* root) {
    if (root == NULL) return;
    printf("%d ", root->key);
    preorder(root->left);
    preorder(root->right);
}

void postorder(Node* root) {
    if (root == NULL) return;
    postorder(root->left);
```

```
    postorder(root->right);
    printf("%d ", root->key);
}

// B3. Height of tree (Convention: empty tree = -1, single node = 0)
int height(Node* root) {
    if (root == NULL) {
        return -1;
    }
    int leftHeight = height(root->left);
    int rightHeight = height(root->right);

    if (leftHeight > rightHeight) {
        return leftHeight + 1;
    } else {
        return rightHeight + 1;
    }
}

// B4. Count total nodes and leaf nodes
int countNodes(Node* root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}

int countLeaves(Node* root) {
    if (root == NULL) return 0;
    // If both children are NULL, this node is a leaf
    if (root->left == NULL && root->right == NULL) {
        return 1;
    }
    return countLeaves(root->left) + countLeaves(root->right);
}

// B5. Find Minimum and Maximum without recursion (Iterative)
Node* findMin(Node* root) {
    if (root == NULL) return NULL;
    // The smallest value is always the leftmost node
    while (root->left != NULL) {
        root = root->left;
    }
    return root;
}

Node* findMax(Node* root) {
    if (root == NULL) return NULL;
    // The largest value is always the rightmost node
    while (root->right != NULL) {
```

```
        root = root->right;
    }
    return root;
}

// PART C: Deletion and Memory Cleanup

Node* deleteNode(Node* root, int key) {
    if (root == NULL) return NULL;

    // Step 1: Locate the node to be deleted
    if (key < root->key) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->key) {
        root->right = deleteNode(root->right, key);
    } else {
        // We found the node, Handle the 3 deletion cases explicitly:

        // Case 1: Leaf node (no children)
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        // Case 2a: One child (only right child exists)
        else if (root->left == NULL) {
            Node* temp = root->right;
            free(root);
            return temp;
        }
        // Case 2b: One child (only left child exists)
        else if (root->right == NULL) {
            Node* temp = root->left;
            free(root);
            return temp;
        }
        // Case 3: Two children
        else {
            // Find inorder successor (smallest node in the right subtree)
            Node* successor = findMin(root->right);
            // Copy the successor's key into this current node
            root->key = successor->key;
            // Recursively delete the successor from the right subtree
            root->right = deleteNode(root->right, successor->key);
        }
    }
    return root;
}
```

```c
// Free all nodes using postorder traversal (children freed before parent)
void freeTree(Node* root) {
    if (root == NULL) return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}

// PART D: Why Balance Matters

// D1. Verify BST property across the entire tree using valid [min, max] range
int isBST(Node* root, int min, int max) {
    if (root == NULL) return 1;

    // If this node's key falls outside the valid range, it violates the BST
rule
    if (root->key <= min || root->key >= max) {
        return 0;
    }

    // Check left subtree (max allowed is root->key) and right subtree (min
allowed is root->key)
    return isBST(root->left, min, root->key) &&
           isBST(root->right, root->key, max);
}

int main() {
    // --- PART A & B ---
    Node* root = NULL;
    int keys[] = {50, 30, 70, 20, 40, 60, 80, 35};
    int n = 8;

    for (int i = 0; i < n; i++) {
        root = insert(root, keys[i]);
    }

    printf("Inorder: "); inorder(root); printf("\n");
    printf("Preorder: "); preorder(root); printf("\n");
    printf("Postorder: "); postorder(root); printf("\n");

    printf("Height: %d Nodes: %d Leaves: %d\n",
           height(root), countNodes(root), countLeaves(root));

    printf("Search 60: %s ", search(root, 60) ? "found" : "not found");
    printf("Search 25: %s ", search(root, 25) ? "found" : "not found");
    printf("Search 90: %s\n", search(root, 90) ? "found" : "not found");

    Node* minNode = findMin(root);
```

```c
    Node* maxNode = findMax(root);
    printf("Min: %d Max: %d\n\n", minNode->key, maxNode->key);

    // --- PART C: DELETION ---
    printf("--- Part C: Deletions ---\n");

    // 1. Delete 35 (Leaf node case)
    root = deleteNode(root, 35);
    printf("Inorder after deleting 35 (Leaf): ");
    inorder(root); printf("\n");
    root = insert(root, 35);

    // 2. Delete 40 (One child case - 40 has left child 35)
    root = deleteNode(root, 40);
    printf("Inorder after deleting 40 (One child): ");
    inorder(root); printf("\n");

    // 3. Delete 30 (Two children case - 30 has left child 20 and right child
35)
    root = deleteNode(root, 30);
    printf("Inorder after deleting 30 (Two children): ");
    inorder(root); printf("\n\n");

    // --- PART D: BALANCE & VALIDATION ---
    printf("--- Part D: Validation & Balance ---\n");
    printf("Is our main tree a valid BST? %s\n",
           isBST(root, INT_MIN, INT_MAX) ? "YES" : "NO");

    // Build the invalid tree from D1 manually
    Node* invalidTree = createNode(50);
    invalidTree->left = createNode(30);
    invalidTree->right = createNode(70);
    invalidTree->left->left = createNode(20);
    invalidTree->left->right = createNode(55); // Invalid! 55 > 50, sitting in
50's left subtree

    printf("Is the manual tree (with 55) a valid BST? %s\n",
           isBST(invalidTree, INT_MIN, INT_MAX) ? "YES" : "NO");
    freeTree(invalidTree);

    // D2: Height of 1 to 15 in sorted order
    Node* sortedTree = NULL;
    for (int i = 1; i <= 15; i++) {
        sortedTree = insert(sortedTree, i);
    }
    printf("Height of 1..15 (Sorted order): %d\n", height(sortedTree));
    freeTree(sortedTree);
```

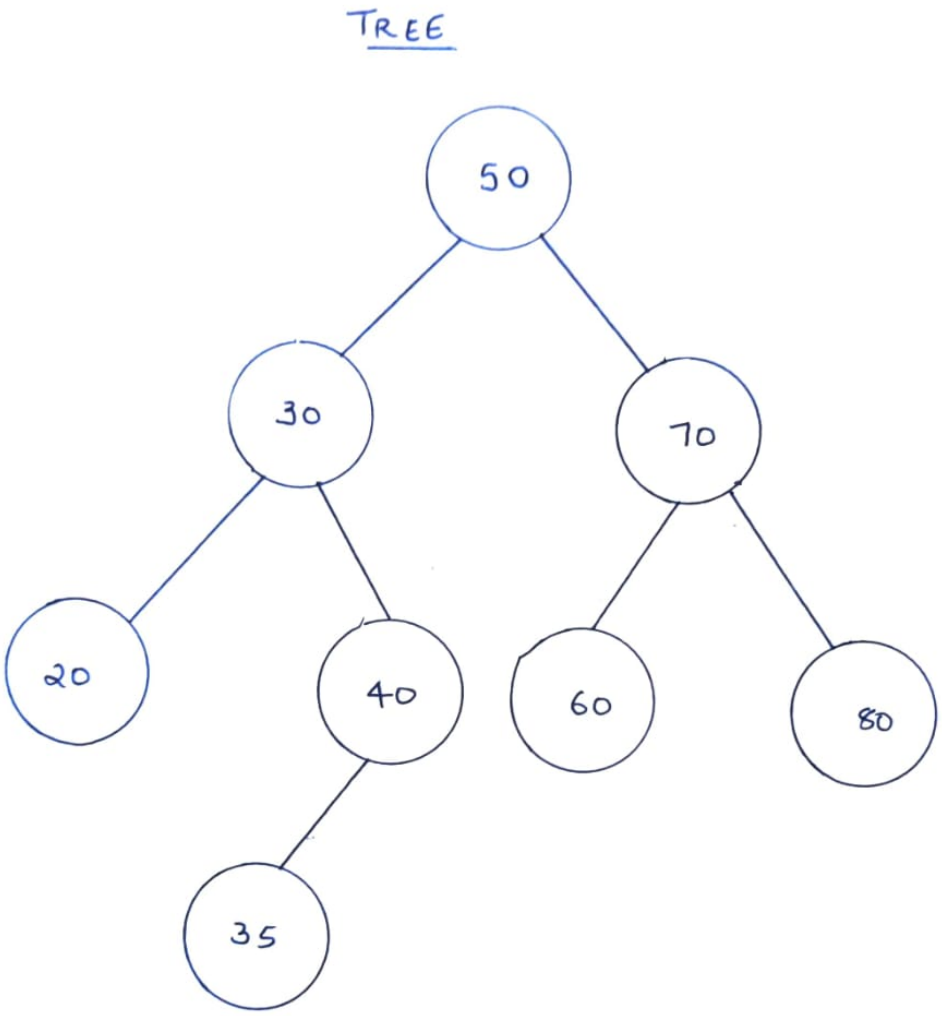
```
    // D2: Height of 1 to 15 in balanced order
    Node* balancedTree = NULL;
    int balKeys[] = {8, 4, 12, 2, 6, 10, 14, 1, 3, 5, 7, 9, 11, 13, 15};
    for (int i = 0; i < 15; i++) {
        balancedTree = insert(balancedTree, balKeys[i]);
    }
    printf("Height of 1..15 (Balanced order): %d\n", height(balancedTree));
    freeTree(balancedTree);

    // D3: Extension - 1000 random integers
    Node* randomTree = NULL;
    srand(time(NULL));
    for (int i = 0; i < 1000; i++) {
        randomTree = insert(randomTree, rand() % 100000);
    }
    printf("Height of 1000 random integers: %d (log2(1000) is ~9.97)\n",
height(randomTree));
    freeTree(randomTree);

    // Free the main tree before exiting
    freeTree(root);

    return 0;
}
```

1. **Your hand-drawn tree from A4.**



2. Your answers to B2, D1, and D2.

B2. Predict the inorder output for the tree in Part A before you run it. What do you notice about it? State the property in one sentence.

Prediction: 20 30 35 40 50 60 70 80

Property in one sentence: Performing an inorder traversal on any valid Binary Search Tree always produces the keys in strictly ascending sorted order.

D1. Write int isBST(Node* root, int min, int max) that verifies the BST property over the whole tree, not just parent-child pairs. Test it on a valid tree and on the invalid tree below, which you build by setting the pointers manually:

50

**/ **

30 70

**/ **

20 55

Every parent-child pair looks fine here (20 < 30 < 55), yet 55 sits in the left subtree of 50 and breaks the rule. Explain why checking only left->key < key < right->key is not enough.

Checking only immediate parent-child pairs is insufficient because the BST property applies to *entire subtrees*, not just direct children. In the manual tree (50 -> left 30 -> right 55), node 30 looks locally valid because 20 < 30 < 55. However, 55 is inside the left subtree of root 50, where every single value must be strictly less than 50. Because 55 is greater than its ancestor 50, the tree is invalid. Passing down min and max bounds ensures every node respects all of its ancestors.

D2. Insert the keys 1 to 15 in sorted order, then print the height. Next insert the same 15 keys in this order and print the height again:

8, 4, 12, 2, 6, 10, 14, 1, 3, 5, 7, 9, 11, 13, 15

Record both heights in a table, and write two or three sentences on what this means for the cost of search.

Insertion Order	Keys Inserted	Resulting Tree Height
Sorted Order	1, 2, 3, ..., 15	14
Balanced Order	8, 4, 12, 2, 6, 10, 14, 1, 3, 5, 7, 9, 11, 13, 15	3

What this means for search cost: The number of comparisons required to search for a key in a BST is bounded by its height, giving a time complexity of O(h). When keys are inserted in sorted order, the tree degenerates into a skewed chain of height n – 1 (height 14), slowing search down to O(n) linear time—no better than a linked list. Inserting in a balanced order keeps the height at $\lfloor \log_2 n \rfloor$ (height 3), ensuring search runs in fast O(log n) logarithmic time.

2. **The output of your program for Parts A to C.**

Output:

```
PS C:\Users\gokul\OneDrive\Desktop\MCA\SEM 2\DSA\LAB\lab-4> .\bst_lab.exe
Inorder: 20 30 35 40 50 60 70 80
Preorder: 50 30 20 40 35 70 60 80
Postorder: 20 35 40 30 60 80 70 50
Height: 3 Nodes: 8 Leaves: 4
Search 60: found Search 25: not found Search 90: not found
Min: 20 Max: 80

--- Part C: Deletions ---
Inorder after deleting 35 (Leaf): 20 30 40 50 60 70 80
Inorder after deleting 40 (One child): 20 30 35 50 60 70 80
Inorder after deleting 30 (Two children): 20 35 50 60 70 80

--- Part D: Validation & Balance ---
Is our main tree a valid BST? YES
Is the manual tree (with 55) a valid BST? NO
Height of 1..15 (Sorted order): 14
Height of 1..15 (Balanced order): 3
Height of 1000 random integers: 18 (log2(1000) is ~9.97)
```

Your code must compile with gcc -Wall without warnings.

Output:

```
PS C:\Users\gokul> wsl -d Ubuntu
gokul@DESKTOP-KEHFV7I:/mnt/c/Users/gokul$ cd "/mnt/c/Users/gokul/OneDrive/Desktop/MCA/SEM 2/DSA/LAB/Lab-4"
gokul@DESKTOP-KEHFV7I:/mnt/c/Users/gokul/OneDrive/Desktop/MCA/SEM 2/DSA/LAB/Lab-4$ gcc -Wall -g bst_lab.c -o bst_lab
gokul@DESKTOP-KEHFV7I:/mnt/c/Users/gokul/OneDrive/Desktop/MCA/SEM 2/DSA/LAB/Lab-4$ ./bst_lab
Inorder: 20 30 35 40 50 60 70 80
Preorder: 50 30 20 40 35 70 60 80
Postorder: 20 35 40 30 60 80 70 50
Height: 3 Nodes: 8 Leaves: 4
Search 60: found Search 25: not found Search 90: not found
Min: 20 Max: 80

--- Part C: Deletions ---
Inorder after deleting 35 (Leaf): 20 30 40 50 60 70 80
Inorder after deleting 40 (One child): 20 30 35 50 60 70 80
Inorder after deleting 30 (Two children): 20 35 50 60 70 80

--- Part D: Validation & Balance ---
Is our main tree a valid BST? YES
Is the manual tree (with 55) a valid BST? NO
Height of 1..15 (Sorted order): 14
Height of 1..15 (Balanced order): 3
Height of 1000 random integers: 19 (log2(1000) is ~9.97)
```

Checking Memory leakage

```
gokul@DESKTOP-KEHFV7I:/mnt/c/Users/gokul/OneDrive/Desktop/MCA/SEM 2/DSA/LAB/Lab-4$ valgrind --leak-check=full ./bst_lab
==538== Memcheck, a memory error detector
==538== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==538== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==538== Command: ./bst_lab
==538==
Inorder: 20 30 35 40 50 60 70 80
Preorder: 50 30 20 40 35 70 60 80
Postorder: 20 35 40 30 60 80 70 50
Height: 3 Nodes: 8 Leaves: 4
Search 60: found Search 25: not found Search 90: not found
Min: 20 Max: 80

--- Part C: Deletions ---
Inorder after deleting 35 (Leaf): 20 30 40 50 60 70 80
Inorder after deleting 40 (One child): 20 30 35 50 60 70 80
Inorder after deleting 30 (Two children): 20 35 50 60 70 80

--- Part D: Validation & Balance ---
Is our main tree a valid BST? YES
Is the manual tree (with 55) a valid BST? NO
Height of 1..15 (Sorted order): 14
Height of 1..15 (Balanced order): 3
Height of 1000 random integers: 25 (log2(1000) is ~9.97)
==538==
==538== HEAP SUMMARY:
==538==     in use at exit: 0 bytes in 0 blocks
==538==   total heap usage: 1,044 allocs, 1,044 frees, 26,056 bytes allocated
==538==
==538== All heap blocks were freed -- no leaks are possible
==538==
==538== For lists of detected and suppressed errors, rerun with: -s
==538== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```